

Question 1

Soit M la taille de l'espace des mots de passe.

Technique	Pré-calcul (temps)	Pré-calcul (espace)	Attaque (temps)	Attaque (espace)
Force brute	–	–	$O(26^M)$ hachages($\frac{26^M}{2}$ en moyenne)	$O(M)$
Base pré-calculée	$O(26^M)$ ha- chages, plus un tri en $O(26^M \cdot M)$	$O(26^M \cdot (M + n))$	$O(M)$ ou $O(1)$ avec une table de hachages.	$O(26^M \cdot (M + n))$

Soit M la longueur des mots de passe; l'espace contient $N = 26^M$ mots de passe.

Force brute : on hache chaque mots des passes jusqu'à trouver le bon ($\frac{26^M}{2}$ essais en moyenne).

Base pré-calculée : on stocke une fois tous les couples $(h(x), x)$, puis l'attaque est une simple recherche. Le coût est amorti, mais la mémoire est en $O(26^M)$.

Valeurs réalistes : un GPU fait environ 2^{33} hashes/s et $26^M \approx 2^{4,7M}$, donc la force brute reste faisable jusqu'à $M \approx 10$ – 11 (environ 2^{50} essais, environ 36h). Pour la base, avec environ 32 octets par entrée, 1 To contient environ 2^{35} entrées on est donc limité par le stockage vers $M \approx 7$ – 8 .

Question 2

On cherche systématiquement un mot de passe *final* $pass_{x,L}$ parmi les N chaînes stockées, afin de récupérer le $pass_{x,0}$ correspondant. La structure doit donc être indexée sur $pass_{x,L}$ (la clé) et renvoyer $pass_{x,0}$ (la valeur).

On utilise une **table de hachage** (dictionnaire) $pass_{x,L} \rightarrow pass_{x,0}$, qui permet une recherche en $O(1)$ en moyenne.

Sans une telle structure, il faudrait parcourir linéairement les N couples à chaque test de point final ($O(N)$ par recherche), ce qui, répété pour les L valeurs de i et les R tables, ferait perdre tout l'intérêt de la méthode. Une indexation sur $pass_{x,L}$ ramène le coût de chaque test à $O(1)$ (ou $O(\log N)$), l'attaque étant alors dominée par le recalcul des chaînes et non par les recherches.

Question 3

Non, pas forcément. Quand on tombe sur $pass_{x,L}$ après i étapes, ça veut dire que la suite de calculs partie de *inputhash* a rejoint la chaîne x à un moment donné. Mais rien ne dit qu'elle l'a rejointe dès le début.

La fonction r envoie un espace énorme (les hashes) vers un espace plus petit (les mots de passe), donc il y a forcément des collisions : deux hashes différents peuvent donner le même mot de passe. Si $inpuhash$ et $hash_{x,j}$ ont la même image par r , on rejoint la chaîne x à partir de là et on arrive sur $pass_{x,L}$, alors que $inpuhash$ n'a jamais été dans la chaîne.

chaîne x : $pass_0 \rightarrow \dots \rightarrow hash_j \xrightarrow{\quad} pass_{\{j+1\}} \rightarrow \dots \rightarrow pass_L$
 $inpuhash \dashrightarrow$ (r donne le même mot de passe)

Dans ce cas le candidat qu'on reconstruit à partir de $pass_{x,0}$ est $pass_{x,L-i-1}$, dont le hash est $hash_{x,L-i-1} \neq inpuhash$. On appelle ça une fausse alarme. Il faut donc toujours vérifier le candidat en calculant $h(candidat)$ et en comparant avec $inpuhash$.

Question 4

Non, à condition de faire la recherche complètement, c'est-à-dire tester tous les i de 0 à $L-1$ et vérifier tous les candidats trouvés (pas seulement le premier).

Si le mot de passe cherché est $pass_{x,j}$ dans une chaîne x (avec $j \leq L-1$), alors $inpuhash = hash_{x,j}$. En appliquant r on retombe sur $pass_{x,j+1}$, puis en continuant on suit exactement la chaîne x et on arrive sur $pass_{x,L}$ au bout de $i = L-1-j$ étapes. Comme tout est déterministe, on le trouve forcément.

Donc si aucun candidat ne marche, c'est que le mot de passe n'est dans aucune chaîne. Petite exception : les $pass_{x,L}$ eux-mêmes, dont le hash n'est jamais calculé, ne sont pas couverts par la table.

Question 5

Si $pass_{x,i} = pass_{y,j}$ avec $x \neq y$ (collision), alors comme h et r sont déterministes, tout ce qui suit est identique dans les deux chaînes : $pass_{x,i+k} = pass_{y,j+k}$ pour tout k . Les deux chaînes fusionnent.

Conséquences :

- **Place et calcul gaspillés.** Tous les mots de passe après la collision sont présents deux fois. La table couvre donc moins de $N \times L$ mots de passe distincts, alors qu'on a payé le calcul de $N \times L$ hashes.
- **Si $i = j$,** les deux chaînes ont le même point d'arrivée $pass_{x,L} = pass_{y,L}$. On peut le détecter au moment du pré-calcul et jeter une des deux chaînes, mais le calcul a déjà été fait pour rien.
- **Si $i \neq j$,** les points d'arrivée sont différents, donc on ne voit pas la fusion dans la table. Le doublon est invisible et impossible à nettoyer.
- **Plus de fausses alarmes.** Lors d'une attaque, si on tombe sur $pass_{y,L}$, on reconstruit la chaîne y depuis $pass_{y,0}$, mais le mot de passe peut en fait être dans la partie de la chaîne x située avant la fusion. Le candidat est faux et on a perdu du temps de calcul.

Plus N et L sont grands, plus les collisions sont probables (paradoxe des anniversaires), donc plus on veut couvrir de mots de passe, moins la méthode est efficace. C'est le principal défaut des hashes chaînés, et c'est ce que les rainbow tables corrigent en utilisant une fonction de réduction différente à chaque colonne : deux chaînes ne fusionnent alors que si la collision arrive à la même colonne.